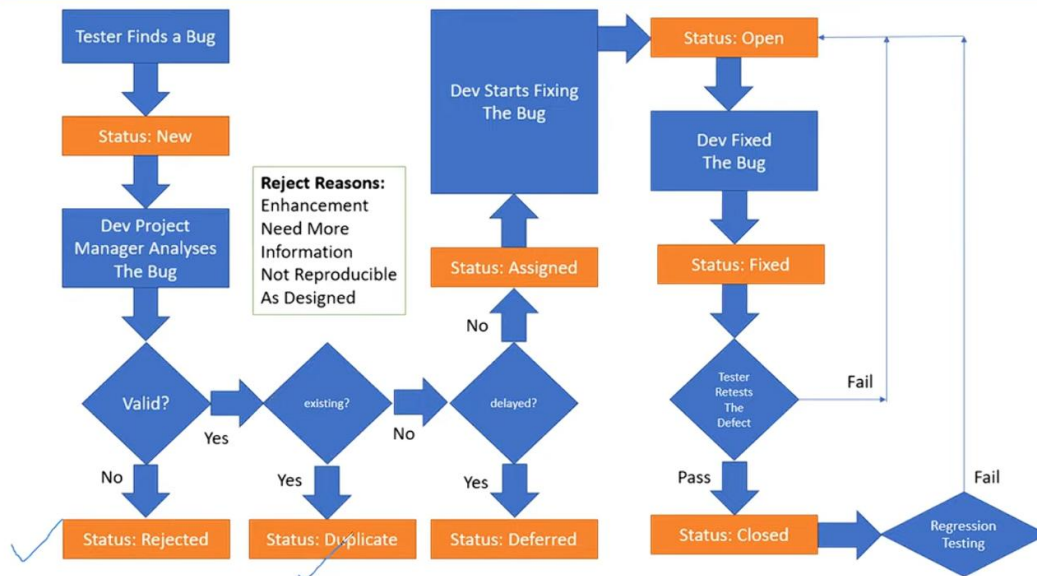


Bug Life Cycle



Test Metrics

SNO	Required Data
1	No. Of Requirements
2	Avg. No. of Test Cases written Per Requirement
3	Total No.of Test Cases written for all Requirement
4	Total No. Of test cases Executed
5	No.of Test Cases Passed
6	No.of Test Cases Failed
7	No.of Test cases Blocked
8	No. Of Test Cases Un Executed
9	Total No. Of Defects Identified
10	Critical Defects Count
11	Higher Defects Count
12	Medium Defects Count
13	Low Defects Count
14	Customer Defects
15	No.of defects found in UAT

Test Metrics

- **% of Test cases Executed:**
 $\text{No. of Test cases executed} / \text{Total No. of Test cases written} \times 100$
- **% of test cases NOT executed:**
 $(\text{No. of Test cases NOT executed} / \text{Total No. of Test cases written}) \times 100$
- **% Test cases passed**
 $(\text{No. of Test cases Passed} / \text{Total Test cases executed}) \times 100$
- **% Test cases failed**
 $(\text{No. of Test cases failed} / \text{Total Test cases executed}) \times 100$
- **% Test cases blocked**
 $(\text{No. of test cases blocked} / \text{Total Test cases executed}) \times 100$

Test Metrics

- **Defect Density: Number of defects identified per requirement/s**
 $\text{No. of defects found} / \text{Size (No. of requirements)}$
- **Defect Removal Efficiency (DRE):**
 $(A / (A+B)) \times 100$
 $(\text{Fixed Defects} / (\text{Fixed Defects} + \text{Missed defects})) \times 100$
 - A- Defects identified during testing/ Fixed Defects
 - B- Defects identified by the customer/Missed defects
- **Defect Leakage:**
 $(\text{No. of defects found in UAT} / \text{No. of defects found in Testing}) \times 100$
- **Defect Rejection Ratio:**
 $(\text{No. of defect rejected} / \text{Total No. of defects raised}) \times 100$
- **Defect Age:** Fixed date-Reported date
- **Customer satisfaction** = No. of complaints per Period of time

7 Principles of Software Testing

1. Start software testing at early stages. Means from the beginning when you get the requirements.
2. Test the software in order to find the defects.
3. Highly impossible to give the bug free software to the customer.
4. Should not do Exhaustive testing. Means we should not use same type of data for testing every time.
5. Testing is context based. Means decide what types of testing should be conducted based on type of application.
6. We should follow the concept of Pesticide Paradox. Means, if you are executing same cases for longer run, they won't find any defects. We have to keep update test cases in every cycle/release in order to find more defects.
7. We should follow defect clustering. Means some of the modules contains most of the defects. By experience, we can identify such risky modules. 80% of the problems are found in 20% of the modules.

1. Defect/Bug Life Cycle

Summary from Transcript

- A defect life cycle defines the states a bug goes through from identification to closure.
- Common statuses: **New** → **Assigned** → **Open** → **Fixed** → **Retested** → **Closed** (or Reopened if issue persists).

Additional Details

- **States Explained:**
 - **New:** Tester reports the defect.
 - **Assigned:** Assigned to a developer.
 - **Open:** Developer starts working on it.
 - **Fixed:** Developer resolves the issue.
 - **Retested:** Tester verifies the fix.
 - **Closed:** Issue resolved successfully.
 - **Reopened:** If the fix fails.
 - **Rejected/Duplicate/Not a Bug:** Special cases.

Diagram:

New → Assigned → Open → Fixed → Retested → Closed

↘ Reopened ↗

Real-world Example:

Login button not working:

- Tester logs defect in JIRA → Status: New
- Assigned to developer → Status: Assigned
- Developer fixes → Status: Fixed
- Tester retests → Status: Closed (or Reopened if still broken)

Interview Importance

- Frequently asked in QA interviews.

Possible Interview Questions:

- Explain defect life cycle.
- What are different defect statuses?
- What happens if a defect is not reproducible?
- Give an example of defect lifecycle in JIRA.

2. Test Closure / When to Stop Testing**Summary from Transcript**

- Testing stops when **exit criteria** are met.
- Includes: All planned test cases executed, no critical defects remain, coverage achieved.

Additional Details**Exit Criteria Examples:**

- 95% test coverage achieved.
- All high-priority defects fixed.
- Test summary report approved.

Diagram:

[All Test Cases Executed] + [Critical Bugs Fixed] + [Coverage Met] → Stop Testing

Real-world Example:

E-commerce app:

- All payment gateway test cases passed.
- No P0/P1 defects open.
- Test report signed off by QA lead.

Interview Importance

- Common theoretical question.

Possible Interview Questions:

- When do you stop testing?
- What is exit criteria?
- Can testing stop if minor bugs exist?

3. Software Testing Metrics

Summary from Transcript

- Metrics measure progress and quality.
- Examples: Test Case Execution %, Defect Density, Defect Removal Efficiency.

Additional Details

Key Metrics:

- **Test Execution %:**
$$\text{Executed Test Cases} \div \text{Total Test Cases} \times 100$$
- **Defect Density:**
$$\text{Defects Found} \div \text{Size of Module (e.g., LOC)}$$
- **Defect Removal Efficiency:**
$$\text{Defects Fixed Before Release} \div \text{Total Defects} \times 100$$

Diagram:

Metrics → Track Progress → Improve Quality

Real-world Example:

- 500 test cases planned, 450 executed → Execution % = 90%.
- 50 defects found in 10K LOC → Defect Density = 0.005.

Interview Importance

- Often asked in managerial rounds.

Possible Interview Questions:

- What are software testing metrics?
 - How do you calculate defect density?
 - Why are metrics important?
-

4. QA/Testing Activities

Summary from Transcript

- Activities include requirement analysis, test planning, test design, environment setup, execution, defect reporting, and closure.

Additional Details

Diagram:

Requirement Analysis → Test Planning → Test Design → Test Execution → Defect Reporting
→ Test Closure

Real-world Example:

Banking app:

- Analyze FRD → Prepare test plan → Write login test cases → Execute → Report bugs
→ Prepare closure report.

Interview Importance

- Common question for freshers.

Possible Interview Questions:

- What are QA activities in STLC?
 - Explain the role of a tester in each phase.
-

5. Principles of Software Testing

Summary from Transcript

- Seven principles:
 1. Testing shows presence of defects.

2. Exhaustive testing is impossible.
3. Early testing saves cost.
4. Defect clustering.
5. Pesticide paradox.
6. Testing is context-dependent.
7. Absence of errors fallacy.

Additional Details

Diagram:

Principles → Guide Testing → Improve Efficiency

Real-world Example:

- **Pesticide Paradox:** Reusing same test cases repeatedly → No new bugs found → Need to update test cases.

Interview Importance

- Very common theoretical question.

Possible Interview Questions:

- Explain pesticide paradox.
- Why is exhaustive testing impossible?
- What is defect clustering?

✅ Quick Revision Table for Interview

Topic	Key Points	Common Questions
Defect Life Cycle	New → Assigned → Fixed → Closed	Explain defect lifecycle.
Test Closure	Exit criteria: coverage, no critical bugs	When do you stop testing?
Testing Metrics	Execution %, Defect Density, DRE	How do you calculate defect density?
QA Activities	Requirement → Planning → Execution → Closure	What are QA activities?

Topic	Key Points	Common Questions
Principles of Testing	7 principles incl. Pesticide paradox	Why exhaustive testing is impossible?